Chapter 21 | Exercise solutions with Python

Image and Video Understanding

This notebook solves Exercises 5–8. It starts by tracing image pixels into patch rows, then separates patch content from position, compares two linear probes, and counts attention scores for three video grids.

The examples run locally with NumPy. They use small images, feature arrays, and clips so that the printed results can be checked against the hand calculations. The written solutions cover Exercises 1–4 and 9–12.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch21_vision"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check row-major patch order

Fill a four-row, six-column image with the integers 0 through 23. Each pixel has one channel. Think of the values as labels for the pixels. They let us trace each entry in a flattened patch back to its original location. A side length of two gives two patch rows and three patch columns. Each patch contributes four values, so the output should have shape $(6, 4)$.

```
from examples.ch21_vision.model import patchify

image = np.arange(24).reshape(4, 6, 1)
patch_rows = patchify(image, patch_size=2)

print("Image values (channel 0)")
print(image[:, :, 0])
print("Flattened patches (one patch per row)")
print(patch_rows)
```

OUTPUT

```
Image values (channel 0)
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]]
Flattened patches (one patch per row)
[[ 0  1  6  7]
 [ 2  3  8  9]
 [ 4  5 10 11]
 [12 13 18 19]
 [14 15 20 21]
 [16 17 22 23]]
```

Row-major means `patchify` visits the three patches across the top before moving to the bottom. It also reads the pixels within each patch across the first row, then across the second. The first output row comes from the upper-left patch, and the last comes from the lower-right patch. With zero-based image coordinates and the single channel fixed at 0, the entries map back as follows.

Output row	Pixel value	Image row	Image column
First	0	0	0
First	1	0	1
First	6	1	0
First	7	1	1
Last	16	2	4
Last	17	2	5
Last	22	3	4
Last	23	3	5

The value at image row r and column c is $6r + c$ because each preceding image row contains six pixels. To check the ordering independently, we can read all six patches directly by their upper-left corners and compare the four entries in each row. A shape check alone would also pass if the patch rows were shuffled.

```
expected_patches = []
for row in (0, 2):
    for column in (0, 2, 4):
        expected_patches.append([
            image[row, column, 0],
            image[row, column + 1, 0],
            image[row + 1, column, 0],
            image[row + 1, column + 1, 0],
        ])

assert patch_rows.shape == (6, 4)
np.testing.assert_array_equal(patch_rows, expected_patches)
```

6. Add a position to an identical patch

Give two patches the same four input values, $(1, 2, 3, 4)$. A projection turns each length-four row into a length-two token by multiplying it by the same weight matrix. We choose the first column of the matrix to add the first and third values, and the second column to subtract the fourth value from the second.

$$(1, 2, 3, 4)\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & -1 \end{bmatrix} = (1 + 3,\ 2 - 4) = (4,\ -2).$$

Each output coordinate is a dot product with one column of the matrix. The two input rows are identical, so both dot products give the same results for each row. Call `patch_embeddings` without a classification token. Both calls therefore return a $(2, 2)$ array, with one row per patch and one column per output coordinate.

```
from examples.ch21_vision.model import patch_embeddings

identical_patches = np.array([[1, 2, 3, 4], [1, 2, 3, 4]])
projection = np.array([[1, 0], [0, 1], [1, 0], [0, -1]])
positions = np.array([[0, 0], [0, 1]])

default_tokens = patch_embeddings(identical_patches, projection)
positioned_tokens = patch_embeddings(
    identical_patches, projection, positions=positions
)

print("Default tokens")
print(default_tokens)
print("Tokens with position vectors")
print(positioned_tokens)
```

```
OUTPUT
Default tokens
[[ 4. -2.]
 [ 4. -2.]]
Tokens with position vectors
[[ 4. -2.]
 [ 4. -1.]]
```

`positions` supplies an additive vector for each token row. Adding $(0, 0)$ leaves the first row at $(4, -2)$. Adding $(0, 1)$ changes the second row to $(4, -1)$.

More generally, let z be the shared projected patch and p_1 and p_2 be any two position vectors of the same width. The returned rows are $z + p_1$ and $z + p_2$. Subtracting the first row from the second cancels z and gives $p_2 - p_1$. Adding the same shift to both positions moves both tokens by that amount and preserves their difference. The exercise chooses the two vectors by hand to make this operation easy to see. In the chapter's Vision Transformer, training learns a position vector for each location.

```
# Check the full arrays against the hand calculation.
np.testing.assert_array_equal(default_tokens, [[4, -2], [4, -2]])
np.testing.assert_array_equal(positioned_tokens, [[4, -2], [4, -1]])
np.testing.assert_array_equal(
    positioned_tokens[1] - positioned_tokens[0], [0, 1]
)
```

7. Compare ordered and sorted features

The four features give the brightness at the upper left, upper right, lower left, and lower right, in that order. Class 0 has its bright patch in the top row and class 1 has it in the bottom row. For example, $(1, 0, 0, 0)$ and $(0, 0, 1, 0)$ place the same brightness in different rows. An ordered head can separate the classes by comparing the sum of the two upper features with the sum of the two lower ones.

Fit a **linear probe**, a classifier that learns weights and biases while keeping the input features fixed. Both fits use four input features, two class scores, 120 training steps and a learning rate of 0.5. The second fit sorts values within each row, preserving which brightness values appear while discarding their locations. The labels stay attached to the same training examples.

```
from examples.ch21_vision.model import train_linear_probe

ordered = np.array([
    [1, 0, 0, 0],
    [0, 0, 1, 0],
    [0, 2, 0, 0],
    [0, 0, 0, 2],
])
labels = np.array([0, 1, 0, 1])
sorted_rows = np.sort(ordered, axis=1)
settings = {"steps": 120, "lr": 0.5}
ordered_weight, ordered_bias = train_linear_probe(
    ordered, labels, **settings
)
sorted_weight, sorted_bias = train_linear_probe(
    sorted_rows, labels, **settings
)

print("Sorted feature rows")
print(sorted_rows)
np.testing.assert_array_equal(
    sorted_rows, [[0, 0, 0, 1], [0, 0, 0, 1],
                  [0, 0, 0, 2], [0, 0, 0, 2]]
)
np.testing.assert_array_equal(labels[::2], 1 - labels[1::2])
```

```
OUTPUT
Sorted feature rows
[[0 0 0 1]
 [0 0 0 1]
 [0 0 0 2]
 [0 0 0 2]]
```

Multiply the feature rows by the fitted weight matrix and add the bias to get two raw scores, or *logits*, for each example. Column 0 scores class 0 and column 1 scores class 1. Choose the larger score on each row. When two scores tie, NumPy's `argmax` returns the first index, which here is class 0.

```
ordered_scores = ordered @ ordered_weight + ordered_bias
sorted_scores = sorted_rows @ sorted_weight + sorted_bias
ordered_predictions = ordered_scores.argmax(axis=1)
sorted_predictions = sorted_scores.argmax(axis=1)

print(
    f"{'Label':>5}  {'Ordered scores':^16}  {'Class':>5}"
    f"  {'Sorted scores':^16}  {'Class':>5}"
)
for row, label in enumerate(labels):
    score0, score1 = ordered_scores[row]
    sorted0, sorted1 = sorted_scores[row]
    print(
        f"{label:>5}  ({score0:6.3f}, {score1:6.3f})"
        f"  {ordered_predictions[row]:>5}"
        f"  ({sorted0:6.3f}, {sorted1:6.3f})"
        f"  {sorted_predictions[row]:>5}"
    )
ordered_correct = np.count_nonzero(ordered_predictions == labels)
sorted_correct = np.count_nonzero(sorted_predictions == labels)
print(f"Ordered correct {ordered_correct} of {len(labels)}")
print(f"Sorted correct  {sorted_correct} of {len(labels)}")

upper = ordered[:, :2].sum(axis=1)
lower = ordered[:, 2:].sum(axis=1)
np.testing.assert_array_equal(ordered_predictions, lower > upper)
np.testing.assert_array_equal(sorted_scores, np.zeros((4, 2)))
np.testing.assert_array_equal(sorted_predictions, [0, 0, 0, 0])
```

OUTPUT

```
Label   Ordered scores   Class   Sorted scores    Class
    0  ( 1.666, -1.666)      0  ( 0.000,  0.000)      0
    1  (-1.666,  1.666)      1  ( 0.000,  0.000)      0
    0  ( 2.387, -2.387)      0  ( 0.000,  0.000)      0
    1  (-2.387,  2.387)      1  ( 0.000,  0.000)      0
Ordered correct 4 of 4
Sorted correct  2 of 4
```

The ordered head predicts $(0, 1, 0, 1)$ and gets all four rows right. In the sorted fit, each distinct input occurs once with each label. The head starts with zero weights and biases, so both classes start with probability $1/2$. The two examples in each pair give equal and opposite updates because their features are identical and their labels differ. The updates cancel at every step, leaving scores $(0, 0)$ on all four rows. The tie rule gives predictions $(0, 0, 0, 0)$ and two correct answers.

The pairwise limit holds for **any deterministic classifier** using only these sorted inputs. A brightness of 1 can still be distinguished from a brightness of 2, but both examples with the same brightness must receive the same prediction. Each pair contains one example of each class, so exactly one prediction per pair is correct regardless of the chosen class.

These are predictions on the four rows used for fitting. The chapter's separate synthetic-image experiment evaluates generalization to held-out brightness and background settings. There the ordered head gets 8 of 8 and the sorted head gets 4 of 8.

8. Trade temporal for spatial tokens (optional)

A token has a frame index and a patch location within that frame. Joint attention can score a query token against every key token in the clip. Divided attention first scores tokens at the same patch location across frames, then scores tokens in the same frame across patch locations.

Count one entry per ordered query and key pair, per attention head. Include a token's score against itself and exclude special tokens. A pair from token A to token B and the reverse pair occupy separate entries. A token's self-score appears once in each of the two divided operations.

With T frames and S patches per frame, the clip has TS tokens. There are S temporal groups of size T and T spatial groups of size S, so the counts are

$$\begin{aligned} \text{Joint} &= (TS)^2, \\ \text{Temporal} &= ST^2, \\ \text{Spatial} &= TS^2, \\ \text{Divided} &= ST^2 + TS^2 = TS(T+S). \end{aligned}$$

For example, $(T, S) = (2, 12)$ has 12 temporal groups with 2^2 entries each and two spatial groups with 12^2 entries each. Check the function by explicitly counting the pairs of token coordinates that each operation scores.

```
from itertools import product

from examples.ch21_vision.model import space_time_pairs

comparisons = []
for frames, patches in [(2, 12), (4, 6), (6, 4)]:
    counts = space_time_pairs(frames, patches)
    tokens = list(product(range(frames), range(patches)))
    pairs = list(product(tokens, repeat=2))
    temporal = sum(
        query[1] == key[1] for query, key in pairs
    )
    spatial = sum(
        query[0] == key[0] for query, key in pairs
    )
    assert len(tokens) == 24
    assert counts == {
        "joint": len(pairs),
        "temporal": temporal,
        "spatial": spatial,
        "divided": temporal + spatial,
    }
    comparisons.append((frames, patches, len(tokens), counts))
```

```
print("Frames  Patches/frame  Tokens  Joint  Temporal  Spatial  Divided")
for frames, patches, total_tokens, counts in comparisons:
    print(
        f"{frames:>6}  {patches:>13}  {total_tokens:>6}"
        f"  {counts['joint']:>5}  {counts['temporal']:>8}"
        f"  {counts['spatial']:>7}  {counts['divided']:>7}"
    )
```

```
OUTPUT
Frames  Patches/frame  Tokens  Joint  Temporal  Spatial  Divided
     2             12      24    576        48      288      336
     4              6      24    576        96      144      240
     6              4      24    576       144       96      240
```

Every clip has 24 tokens and $24^2 = 576$ joint scores. The divided count drops from $24(2 + 12) = 336$ to $24(4 + 6) = 240$ as the group sizes become closer. Swapping 4 and 6 interchanges the temporal and spatial counts, 96 and 144, but their sum stays 240. Both TS and $T + S$ are unchanged by the swap. The two operations still group the tokens along different axes, so the equal totals describe the number of entries each block scores.